

AIAC ASSIGNMENT 10.1

NAME: ISHA MADEEHA

HALLTICKETNO; 2303A51017

BATCH-01

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty

Python script.

Sample Input Code:

```
# Calculate average score of a student

def calc_average(marks):

    total = 0

    for m in marks:

        total += m

    average = total / len(marks)

    return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Corrected code

#refactor the code to fix the typo in the return statement syntax error

```
def calc_average(marks):

    total = 0#` Initialize total to 0`

    for m in marks:# Loop through each mark in the list

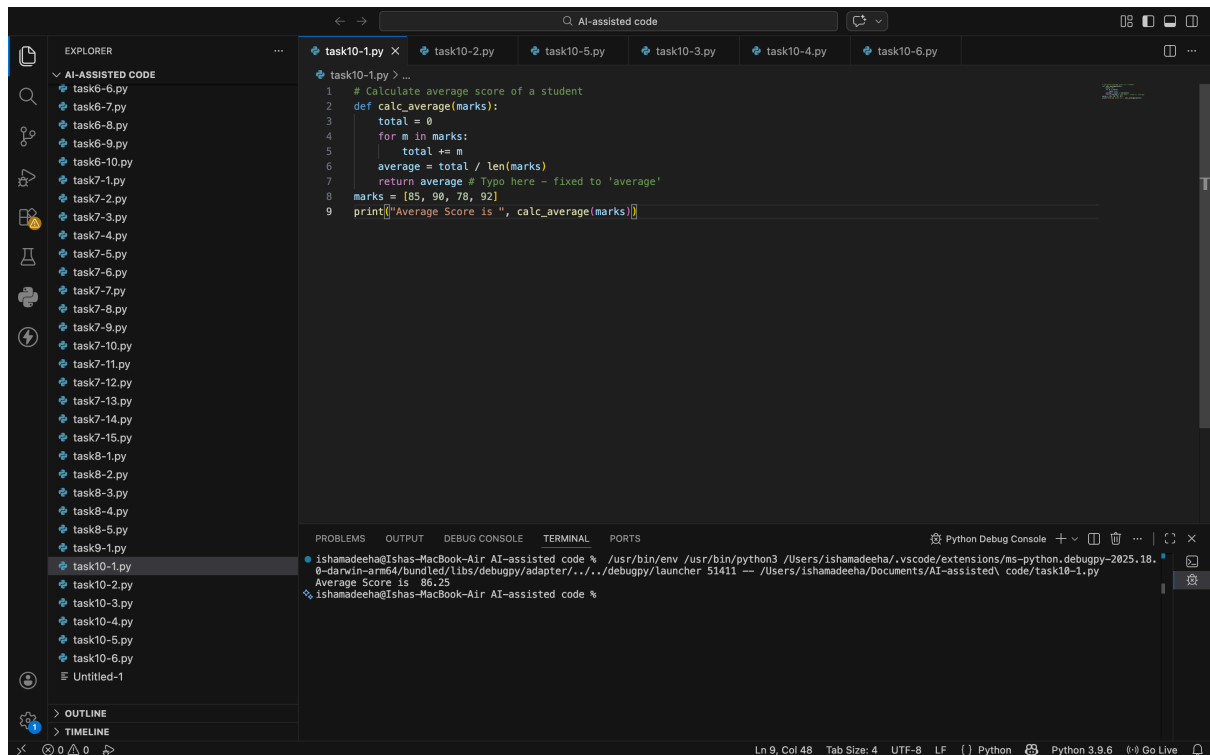
        total += m# Add the mark to the total

    average = total / len(marks)

    return average # Typo here

marks = [85, 90, 78, 92]# Example marks

print("Average Score is ", calc_average(marks))# Call the function and print the average score
```



Indentation Errors

Python requires proper indentation inside functions and loops.

All lines inside `calc_average()` must be indented.

Typo in Variable Name

return avrage → corrected to return average.

Missing Closing Parenthesis

`print("Average Score is ", calc_average(marks)`
added `)` at the end.

Minor Output Formatting

Removed extra space before comma in print for cleaner output

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style

guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
```

```
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

Correct code:

Function to calculate the area of a rectangle

def area_of_rect(L: int, B: int) -> int:

"""

Calculate the area of a rectangle.

Args:

L: Length of the rectangle

B: Breadth of the rectangle

Returns:

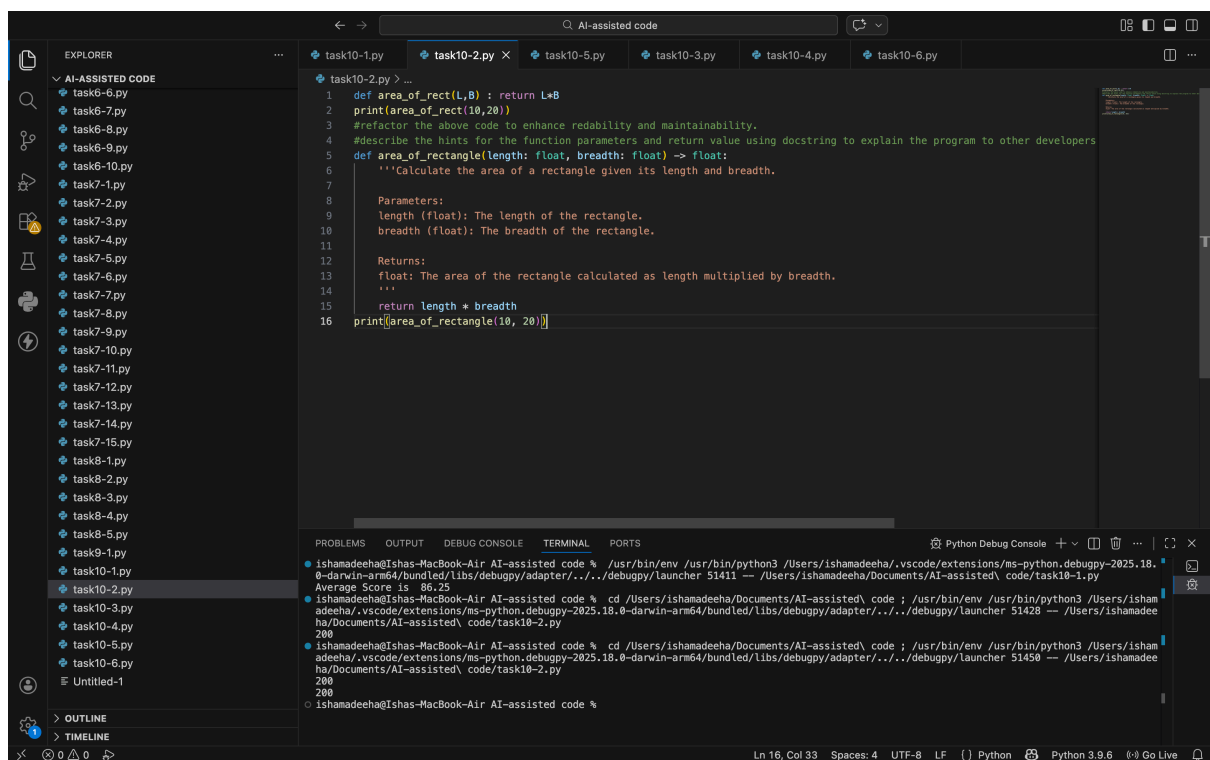
The area of the rectangle

"""

area = L * B

return area

print(area_of_rect(10, 20)) # Call the function and print the area of the rectangle



Function and variable names

Changed L, B → length, breadth (more descriptive, lowercase).

Spacing

Removed extra space before :

Added spaces around operators (*).

Line structure

Avoided one-line function definition.

Added blank line after function (recommended by PEP 8).

Docstring added

Short description improves readability and documentation

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):  
    return x*y/100  
  
a=200  
  
b=15  
  
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting

corrected code

```
def c(x,y):  
    return x*y/100  
  
a=200  
  
b=15  
  
print(c(a,b))
```

refactor the code to fix the indentation error and add a docstring to the function make sure it as more readable and maintainable

```
def calculate_percentage(x: float, y: float) -> float:
```

"""

Calculate the percentage of x with respect to y.

Args:

x: The value for which the percentage is to be calculated

y: The total value that represents 100%

Returns:

The percentage of x with respect to y

"""

return (x * y) / 100# Calculate the percentage of a with respect to b

a = 200# Example value for x

b = 15# Example value for y

print(calculate_percentage(a, b)) # Call the function and print the result

```
1 def c(x,y):
2     return x*y/100
3
4 a=200
5 b=15
6 print(c(a,b))
7
8 #refactor the code to make code more readable without changing its logic with in line comments to explain the code to other
9 def calculate_percentage(part: float, whole: float) -> float:
10     return part * whole / 100 # This function calculates the percentage of 'part' with respect to 'whole' by multiplying 'p
11
12 a = 200
13 b = 15
14 print(calculate_percentage(a, b)) # This line calls the calculate_percentage function with 'a' as the part and 'b' as the w
```

Descriptive function name

c() → calculate_percentage()

Meaningful variable names

a → total_amount

b → percentage

Proper spacing & formatting

Followed Python indentation and spacing rules.

Added comments

Explained function purpose, logic, and inputs

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]  
print("Welcome", students[0])  
print("Welcome", students[1])  
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Corrected code

```
students = ["Alice", "Bob", "Charlie"]  
  
#refactor this code and break it into reusable function and add docstring to the function  
def welcome_students(students: list) -> None:  
    """
```

Print a welcome message for each student in the list.

Args:

students: A list of student names

Returns:

None

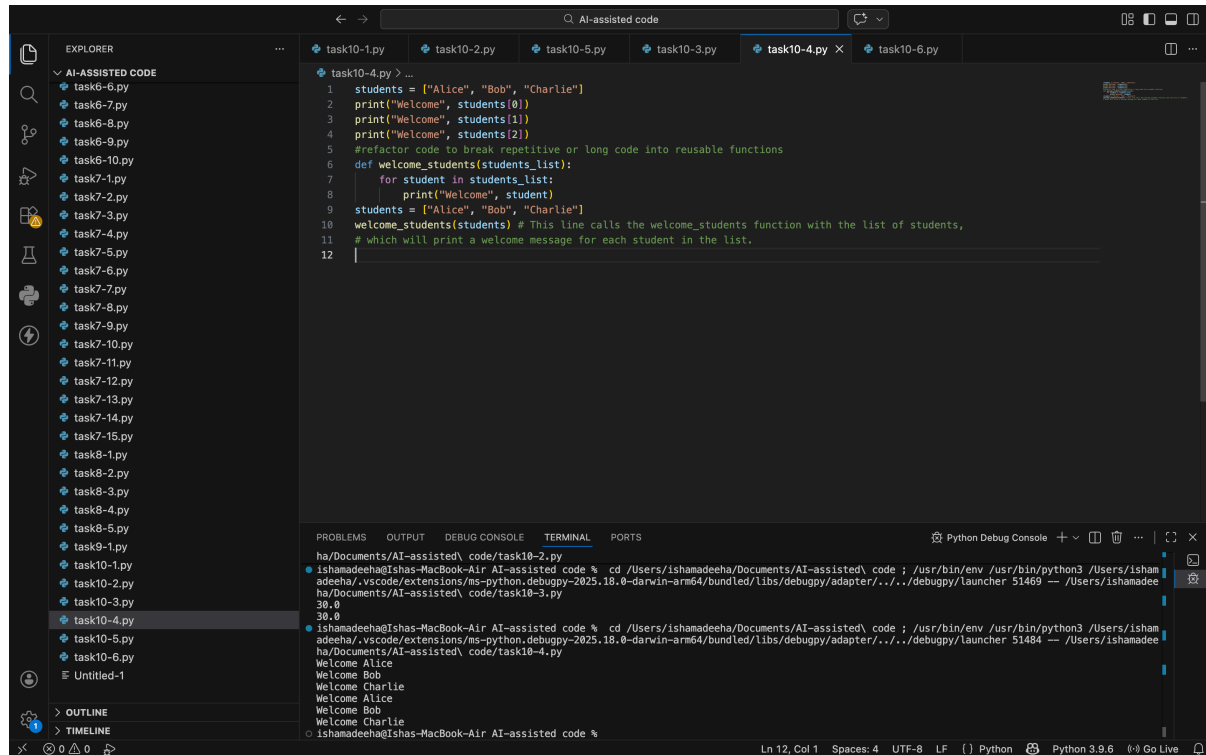
```
    """
```

```
    for student in students:
```

```
        print("Welcome", student)
```

```
students = ["Alice", "Bob", "Charlie"] # Example list of students
```

```
welcome_students(students) # Call the function to welcome the students
```



```
1 students = ["Alice", "Bob", "Charlie"]
2 print("Welcome", students[0])
3 print("Welcome", students[1])
4 print("Welcome", students[2])
5 #refactor code to break repetitive or long code into reusable functions
6 def welcome_students(students_list):
7     for student in students_list:
8         print("Welcome", student)
9 students = ["Alice", "Bob", "Charlie"]
10 welcome_students(students) # This line calls the welcome_students function with the list of students,
11 # which will print a welcome message for each student in the list.
12
```

Python Debug Console

```
ha/Documents/AI-assisted\ code\task10-7.py
ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted\ code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python.debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51469 -- /Users/ishamadeeha/Documents/AI-assisted\ code\task10-3.py
30.0
ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted\ code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python.debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51484 -- /Users/ishamadeeha/Documents/AI-assisted\ code\task10-4.py
Welcome Alice
Welcome Bob
Welcome Charlie
Welcome Alice
Welcome Bob
Welcome Charlie
Welcome Alice
Welcome Charlie
```

Improvements Made

1. Created reusable function

welcome_student(name) avoids repeating print statements.

2. Used loop instead of manual indexing

Works automatically for any number of students.

3. Improved maintainability

If the welcome message changes, update it only in one place.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

```
nums = [i for i in range(1,1000000)]
```

```
squares = []
```

```
for n in nums:
```

```
squares.append(n**2)
```

```
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Corrected code

```
import time# Original code with higher time complexity
```

```
time1 = time.time()
```

```
nums = [i for i in range(1,1000000)]
```

```
squares = []# Calculate squares of numbers
```

```
for n in nums:
```

```
    squares.append(n**2)# Append the square of n to the squares list
```

```
"""
```

```
returns the length of the squares list, which is the same
```

```
as the length of the nums list
```

```
"""
```

```
print(len(squares))
```

```
time2 = time.time()
```

```
print("Time taken: ", time2 - time1)# Refactored code with lower time complexity
```

```
#refactor the code with less time complexity
```

```
time3 = time.time()
```

```
nums = [i for i in range(1,1000000)]
```

```
squares = [n**2 for n in nums]# Calculate squares of numbers using list comprehension, which is more efficient than appending to a list in a loop
```

```
print(len(squares))
```

```
"""
```

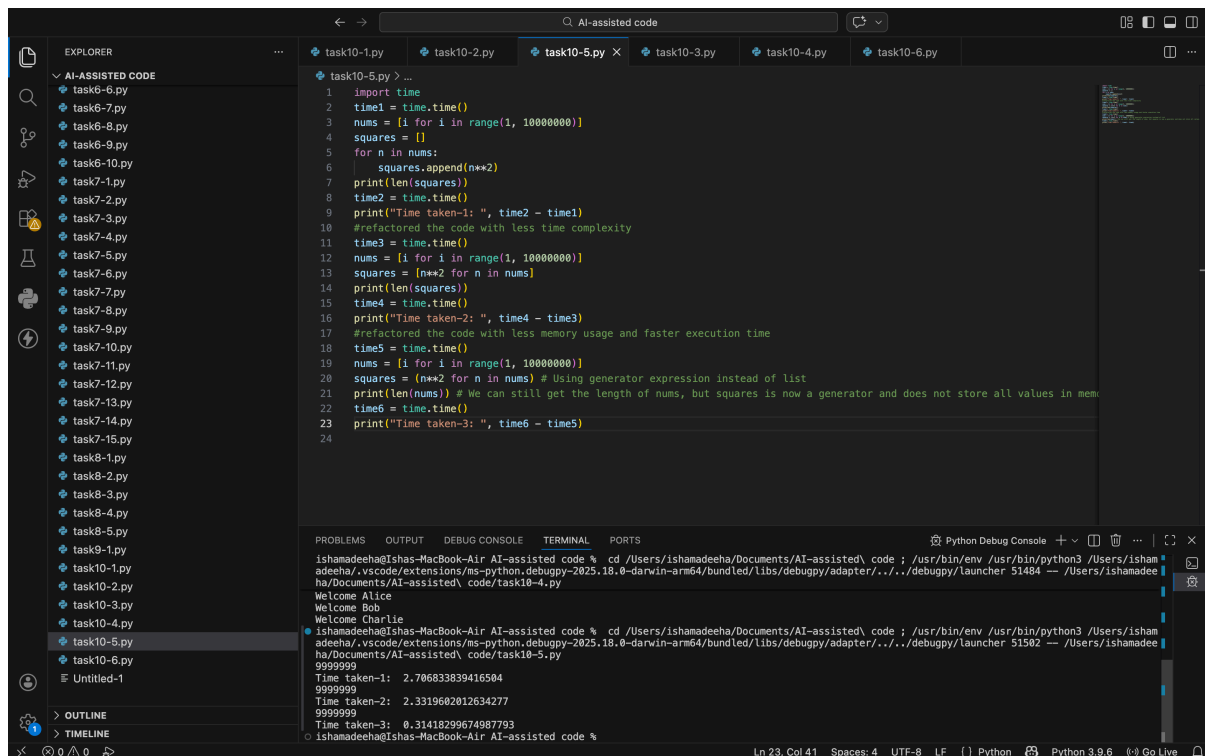
```
returns the length of the squares list, which is the same
```

```
as the length of the nums list
```

```
"""
```

```
time4 = time.time()# Calculate the time taken for the refactored code and print it
```


print("Time taken: ", time4 - time3)# The refactored code should be significantly faster than the original code due to the use of list comprehension, which is optimized for performance in Python.



```
1 import time
2 time1 = time.time()
3 nums = [i for i in range(1, 10000000)]
4 squares = []
5 for n in nums:
6     squares.append(n**2)
7 print(len(squares))
8 time2 = time.time()
9 print("Time taken-1: ", time2 - time1)
10 #refactored the code with less time complexity
11 time3 = time.time()
12 nums = [i for i in range(1, 10000000)]
13 squares = [n**2 for n in nums]
14 print(len(squares))
15 time4 = time.time()
16 print("Time taken-2: ", time4 - time3)
17 #refactored the code with less memory usage and faster execution time
18 time5 = time.time()
19 nums = [i for i in range(1, 10000000)]
20 squares = (n**2 for n in nums) # Using generator expression instead of list
21 print(len(nums)) # We can still get the length of nums, but squares is now a generator and does not store all values in mem
22 time6 = time.time()
23 print("Time taken-3: ", time6 - time5)
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted\ code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python.debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51484 -- /Users/ishamadeeha/Documents/AI-assisted\ code/task10-5.py

Welcome Alice
Welcome Bob
Welcome Charlie

ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted\ code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python.debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 51502 -- /Users/ishamadeeha/Documents/AI-assisted\ code/task10-5.py

9999999
Time taken-1: 2.706833839416584
9999999
Time taken-2: 2.3319602012634277
9999999
Time taken-3: 0.31418299674987793

ishamadeeha@Ishas-MacBook-Air AI-assisted code %

Why This Is Faster

- Avoided creating a full list unnecessarily**
range() is memory efficient (lazy iterable).
- Used list comprehension**
Faster than repeatedly calling .append() in a loop.
- Cleaner and shorter code**
Improves both speed and readability

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
```

```
    if score >= 90:
```

```
        return "A"
```

```
    else:
```

```
        if score >= 80:
```

```
return "B"

else:

if score >= 70:

return "C"

else:

if score >= 60:

return "D"

else:

return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Corrected code

simple function to calculate grade based on score

```
def grade(score):
```

```
    if score >= 90:
```

```
        return 'A'
```

```
    elif score >= 80:
```

```
        return 'B'
```

```
    elif score >= 70:
```

```
        return 'C'
```

```
    elif score >= 60:
```

```
        return 'D'
```

```
    else:
```

```
        return 'F'
```

regenerate this code in simple and more readable way and add docstring to the function

```
def calculate_grade(score: float) -> str:
```

```
    """
```

```
    Calculate the grade based on the given score.
```

Args:

score: The score for which the grade is to be calculated

Returns:

The grade corresponding to the score

"""

if score >= 90:

return 'A' # Grade A for scores 90 and above

elif score >= 80:

return 'B' # Grade B for scores between 80 and 89

elif score >= 70:

return 'C' # Grade C for scores between 70 and 79

elif score >= 60:

return 'D' # Grade D for scores between 60 and 69

else:

return 'F' # Grade F for scores below 60

Example usage

score = 85 # Example score

print("Grade for score", score, "is", calculate_grade(score)) # Call the function and print the grade

```
1 def grade(score):
2     if score >= 90:
3         return "A"
4     else:
5         if score >= 80:
6             return "B"
7         else:
8             if score >= 70:
9                 return "C"
10            else:
11                if score >= 60:
12                    return "D"
13                else:
14                    return "F"
15 print(grade(85)) # Output: B
16 #refactor the code to simplify overly complex logic and improve readability by using elif statements instead of nested if-e
17 def grade(score):
18     if score >= 90:
19         return "A"
20     elif score >= 80:
21         return "B"
22     elif score >= 70:
23         return "C"
24     elif score >= 60:
25         return "D"
26     else:
27         return "F"
28 print(grade(85)) # Output: B
29
```

